

Case Study 01 — Theoretical Only

Canonical TD-DFT-only workflow

Contents

1	Case Study 01 — Theoretical Only	1
1.1	What you will produce	1
1.2	1. Prerequisites	1
1.3	2. Inputs	2
1.4	3. Run with the CLI	2
1.5	4. Run with the HTTP API	3
1.6	5. Drive both interfaces from one Python script	3
1.7	6. Expected output layout	3
1.8	7. Related case studies	3

1 Case Study 01 — Theoretical Only

Run `overlap-calculator` on Gaussian TD-DFT outputs only, with no measured spectra in the manifest. This is the canonical theoretical workflow and the place to start if this is your first case study — the prerequisites section here is referenced from every other case.

1.1 What you will produce

Everything is written under one `output/` directory inside this case study folder:

Folder	By	Contents
<code>output/cli/tables/</code>	CLI	Result tables (CSV / JSON / XLSX)
<code>output/cli/plots/</code>	CLI	TIFF plots per sample, light source, and (sample, light) pair
<code>output/api/</code>	API	Same <code>tables/</code> and <code>plots/</code> tree, extracted from the ZIP
<code>output/api/analysis_outputs.zip</code>	API	Raw ZIP response

Light sources used: **AM1.5G** + **LED-B4**.

1.2 1. Prerequisites

Follow the steps below in order. The first three steps install the package; the last two start and verify the HTTP API.

1.2.1 1.1 Install the package (pick one)

Conda (recommended). From the repository root, run each command in a terminal:

```
conda env create -f environment.yml
conda activate overlap-calculator
pip install -e .
```

Or, plain venv. From the repository root:

```
python -m venv .venv
```

Activate it. Linux / macOS:

```
source .venv/bin/activate
```

Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

Then install the package in editable mode:

```
pip install -e .
```

Both paths put the `overlap-calculator` command on your PATH. Both the CLI and the API run from the same Python environment.

1.2.2 1.2 Install the helper script dependency

The Python helper `submit.py` talks to the HTTP API, which needs the `requests` package:

```
pip install requests
```

1.2.3 1.3 Start the HTTP API

In a **separate** terminal at the repository root, start the API:

```
python -m overlap_calculator.api.app
```

The API now listens on `http://localhost:8000`. Leave that terminal running for the rest of the case study.

If you prefer Docker, run instead:

```
docker compose up -d
```

That publishes the API on host port 8080. Tell `submit.py` to target the Docker port:

```
export OVERLAP_API_URL=http://localhost:8080/analyze
```

Windows PowerShell equivalent:

```
$env:OVERLAP_API_URL = "http://localhost:8080/analyze"
```

1.2.4 1.4 Verify the API is reachable

In any terminal:

```
curl http://localhost:8000/health
```

You should see:

```
{"status": "ok"}
```

If you started the API via `docker compose`, swap port 8000 for 8080 in the `curl` URL.

1.3 2. Inputs

The case-study `input.json` lists 2 Gaussian TD-DFT outputs. The manifest uses relative paths that point at the bundled data under `../input/files/`, so no files are copied or duplicated. To swap in your own data, replace `input.json` with a manifest of the same shape; you do not need to touch `submit.py`.

1.4 3. Run with the CLI

Open a terminal at the repository root, then run a single command:

```
overlap-calculator analyze --input case_studies/01_theoretical_only/input.json --out
↪ case_studies/01_theoretical_only/output/cli --default-light-sources AM15G,LEDB4 --plot-dpi 400
```

After the run, `case_studies/01_theoretical_only/output/cli/` contains a `tables/` directory (and a `plots/` directory unless plots were disabled). The `--default-light-sources` flag pins the bundled sources to AM1.5G + LED-B4; remove it to compare against the full default set (AM15G,LEDB4,LEDB2,LEDB3,CIEFL10).

1.5 4. Run with the HTTP API

Start the API as documented in [Case Study 01 §1](#) and verify with `curl http://localhost:8000/health`. Then submit the same uploads in one curl command:

```
curl -X POST http://localhost:8000/analyze -F "files=@input/files/slurm-5473089.out" -F
↪ "files=@input/files/slurm-5473178.out" -F "default_light_sources=AM15G,LEDB4" -F "plot_dpi=400"
↪ --output case_studies/01_theoretical_only/output/api/analysis_outputs.zip
```

Each `-F "files=@..."` is one upload. The form fields mirror the CLI flags. The response is a ZIP archive saved as `output/api/analysis_outputs.zip`.

1.6 5. Drive both interfaces from one Python script

Instead of typing the commands above, run `submit.py` from the repository root:

```
python case_studies/01_theoretical_only/submit.py
```

The script reads `input.json`, runs the CLI step (writing to `output/cli/`), then POSTs the same uploads to `/analyze` and unpacks the response into `output/api/`. If the API is not reachable, it prints a hint and exits cleanly without failing the CLI step.

1.7 6. Expected output layout

```
case_studies/01_theoretical_only/output/
+-- cli/
|   +-- tables/results.{csv,json,xlsx}          (2 samples × 2 light sources = 4 rows)
|   +-- tables/results_timings.{csv,json,xlsx}
|   +-- tables/descriptor_summary.{csv,xlsx}
|   +-- tables/ranking_by_light_source_<metric>.{csv,json,xlsx}  (gaussian/lorentzian x
↪ absorbed_fraction/shape_overlap)
|   +-- tables/ranking_by_sample_<metric>.{csv,json,xlsx}        (same four metric variants)
|   +-- plots/                                                    (also
↪ plots/ranking/by_light_source/<metric>/ and plots/ranking/by_sample/<metric>/)
+-- api/
    +-- analysis_outputs.zip
    +-- tables/...
    +-- plots/...
```

API sample IDs are auto-generated from each upload's `%chk` header, while the CLI uses the `sample_id` you set in `input.json`. This is the only cosmetic difference between the two trees; the tables align row by row otherwise.

1.8 7. Related case studies

- [Case Study 02 — Experimental Only](#)
- [Case Study 03 — Mixed Theory + Experiment](#)
- [Case Study 06 — Plot DPI](#)
- [Case Study 08 — Broadening Sigma](#)